1) MapToInt() Operation:- (More Detailed)

In Java Streams, **mapToInt()** is an *intermediate operation* that converts a stream of objects into an IntStream (a specialized stream for primitive int values).

🔑 **Why mapToInt() Exists**

- Regular Stream<T> works with objects, not primitives.
- If you want to perform numeric operations (like sum, average, min, max), it's more efficient to work with primitive streams (IntStream, LongStream, DoubleStream).
- mapToInt() bridges the gap: it takes each element of the stream and maps it to an int.

⚡ **Example 1: Mapping Strings to Their Lengths**

```
import java.util.Arrays;


public class Demo {
  public static void main(String[] args) {
    String[] names = {"Alice", "Bob", "Charlie"};


    int totalLength = Arrays.stream(names)
                    .mapToInt(String::length) // convert each String to int // {5, 3, 7}
                    .sum();            // sum all lengths


    System.out.println("Total length: " + totalLength); // Output: 15
  }
}
```

⚡ **Example 2: Mapping Objects to a Numeric Field**

```
import java.util.List;


class Product {
  String name;
```

```
    int price;
    Product(String name, int price) { this.name = name; this.price = price; }
}

public class Demo {
    public static void main(String[] args) {
        List<Product> products = List.of(
            new Product("Book", 200),
            new Product("Pen", 50),
            new Product("Bag", 500)
        );

        int totalPrice = products.stream()
                        .mapToInt(p -> p.price) // extract price field
                        .sum();

        System.out.println("Total price: " + totalPrice); // Output: 750
    }
}
```

🎯 **Key Points**

- **Intermediate function**: mapToInt() transforms the stream but doesn't consume it.
- Produces an IntStream, which has specialized methods like sum(), average(), max(), min().
- Commonly used when you need to **aggregate numeric data** from objects.

👉 In short: mapToInt() is the go-to tool when you want to turn a stream of objects into a stream of integers for efficient numeric operations.

2) Max () Operation: -

In Java Streams, max() is a **terminal operation** — meaning it consumes the stream and produces a single result (not another stream). It's used to find the maximum element according to a comparator or natural ordering.

🔑 **How max() Works**

- Operates on a Stream<T> or primitive streams (IntStream, LongStream, DoubleStream).
- Returns an Optional<T> (or OptionalInt, OptionalLong, OptionalDouble for primitive streams).
- Requires either:
  - Natural ordering (elements must implement Comparable).
  - A custom Comparator.

⚡ **Example 1: Natural Ordering**

java

```
import java.util.Arrays;

import java.util.List;

import java.util.Optional;


public class Demo {

  public static void main(String[] args) {

    List<Integer> numbers = Arrays.asList(10, 25, 3, 42);


    Optional<Integer> maxNum = numbers.stream().max(Integer::compare);


    maxNum.ifPresent(System.out::println); // Output: 42

  }

}
```

⚡ **Example 2: Custom Comparator**

java

```
import java.util.*;

class Product {
  String name;
  int price;
  Product(String name, int price) { this.name = name; this.price = price; }
}

public class Demo {
  public static void main(String[] args) {
    List<Product> products = List.of(
      new Product("Book", 200),
      new Product("Pen", 50),
      new Product("Bag", 500)
    );

    Optional<Product> maxPriceProduct = products.stream()
      .max(Comparator.comparingInt(p -> p.price));

    maxPriceProduct.ifPresent(p ->
      System.out.println("Most expensive: " + p.name + " - " + p.price));
    // Output: Most expensive: Bag - 500
  }
}
```

⚡ **Example 3: Primitive Streams**

java

```
import java.util.stream.IntStream;
```

```
public class Demo {
  public static void main(String[] args) {
    int max = IntStream.of(5, 12, 8, 20).max().orElse(-1);
    System.out.println("Max value: " + max); // Output: 20
  }
}
```

🎯 **Key Points**

- **Terminal operation**: max() ends the stream pipeline and produces a single result.
- Always returns an Optional because the stream might be empty.
- Efficient for finding maximum values in collections or arrays.

👉 In short: max() is the terminal operation you use when you want to extract the **largest element** from a stream, either by natural order or a custom comparator.

3) Min () Operation: -

In Java Streams, min() is a **terminal operation** that finds the smallest element in the stream, either by natural ordering or using a custom comparator. Like max(), it consumes the stream and returns an Optional.

🔑 **How min() Works**

- Operates on Stream<T> or primitive streams (IntStream, LongStream, DoubleStream).
- Returns an Optional<T> (or OptionalInt, OptionalLong, OptionalDouble for primitive streams).
- Requires elements to be comparable or a comparator to be provided.

⚡ **Example 1: Natural Ordering**

java

```
import java.util.Arrays;

import java.util.List;

import java.util.Optional;


public class Demo {

  public static void main(String[] args) {

    List<Integer> numbers = Arrays.asList(10, 25, 3, 42);


    Optional<Integer> minNum = numbers.stream().min(Integer::compare);


    minNum.ifPresent(System.out::println); // Output: 3

  }

}
```

⚡ **Example 2: Custom Comparator**

java

```
import java.util.*;
```

```
class Product {
  String name;
  int price;
  Product(String name, int price) { this.name = name; this.price = price; }
}

public class Demo {
  public static void main(String[] args) {
    List<Product> products = List.of(
      new Product("Book", 200),
      new Product("Pen", 50),
      new Product("Bag", 500)
    );

    Optional<Product> cheapestProduct = products.stream()
      .min(Comparator.comparingInt(p -> p.price));

    cheapestProduct.ifPresent(p ->
      System.out.println("Cheapest: " + p.name + " - " + p.price));
    // Output: Cheapest: Pen - 50
  }
}
```

⚡ **Example 3: Primitive Streams**

java

```
import java.util.stream.IntStream;

public class Demo {
  public static void main(String[] args) {
```

```
    int min = IntStream.of(5, 12, 8, 20).min().orElse(-1);

    System.out.println("Min value: " + min); // Output: 5

  }

}
```

🎯 **Key Points**

- **Terminal operation**: min() ends the stream pipeline and produces a single result.
- Always returns an Optional because the stream might be empty.
- Useful for finding the smallest element in collections, arrays, or custom objects.

👉 In short: min() is the mirror of max() — it extracts the **smallest element** from a stream, either by natural order or a comparator.

4) Then, why it returns the max element when min((Integer val1, Integer val2) -> val2 - val1)?

Excellent observation — the reason your min() call is returning the **maximum element** lies in the **comparator logic** you wrote.

🔑 **How Comparators Work**

- min(comparator) finds the element that is considered "smallest" according to the comparator.
- A comparator compares two values (val1, val2) and returns:
  - Negative → val1 is considered smaller.
  - Zero → equal.
  - Positive → val1 is considered larger.

⚡ **Your Code**

min((Integer val1, Integer val2) -> val2 - val1)

- Normally, you'd write val1 - val2 to compare in ascending order.
- But here you wrote val2 - val1, which **reverses the order**.
- That means the comparator treats larger numbers as "smaller."
- So when you call min(), it actually picks the maximum element, because in this reversed ordering, the maximum is considered the "minimum."

✅ **Correct Usage**

- Ascending order (true minimum):

```
numbers.stream().min((a, b) -> a - b); // returns smallest
```

- Descending order (true maximum):

```
numbers.stream().min((a, b) -> b - a); // returns largest
```

- Or simply use max() with a normal comparator: (preferred)

```
numbers.stream().max((a, b) -> a - b); // returns largest
```

🎯 **Key Insight**

- min() **and** max() **don't inherently know about "smallest" or "largest."**
- They rely entirely on the comparator you provide.
- If you flip the comparator logic, you flip the meaning of "min" and "max."

👉 In short: your min((val1, val2) -> val2 - val1) returns the maximum because you reversed the comparator, making the largest element appear as the "minimum."

5) Method Reference Operator[: :]

The symbol :: in Java is called the **method reference operator** (sometimes informally referred to as the **scope resolution operator**).

🔑 **What It Does**

- It provides a shorthand to refer to methods or constructors **without actually calling them**.
- Instead of writing a full lambda expression, you can point directly to an existing method.
- The compiler resolves which method to use based on the context (parameters and return type).

⚡ **Types of Method References**

1. **Static method reference**

```
Function<String, Integer> func = Integer::parseInt;

System.out.println(func.apply("123")); // Output: 123
```

2. **Instance method of a particular object**

```
String str = "hello";

Supplier<Integer> supplier = str::length;

System.out.println(supplier.get()); // Output: 5
```

3. **Instance method of an arbitrary object of a type**

```
List<String> names = List.of("Alice", "Bob", "Charlie");

names.forEach(System.out::println); // Prints each name
```

4. **Constructor reference**

```
Supplier<List<String>> listSupplier = ArrayList::new;

List<String> list = listSupplier.get();
```

🎯 **Why It's Useful**

- Makes code **shorter and cleaner** compared to lambdas.
- Improves readability by directly pointing to existing methods.
- Commonly used in **Streams API** for operations like map, filter, forEach.

👉 So, the operator :: is officially called the **method reference operator** in Java. It's essentially a way to "resolve" a method or constructor by name and pass it around as a function.